Facultad de Ingeniería y Ciencias
UNIVERSIDAD ADOLFO IBÁÑEZ
Diplomado en Hidrología e
Impactos del Cambio Climático

Guía de modelos Generación por k-NN:

Esta guía de estudio tiene como objetivo entender mejor el método de generación de series hidroclimáticas por k-NN

1) Genere un código de generación de k-NN, que parta de datos periódicos a nivel diario y le permitan generar series sintéticas hidroclimáticas.
2) Importar datos de los archivos:
   - tmax_cr2met_day.csv
   - tmin_cr2met_day.csv
   - precip_cr2met_day.csv

   Note que los archivos importados con de la cuenca Zanjón de la Aguada Puente Particular.
3) Utilizando el código elaborado en la pregunta "1)" genere al menos 20 años (ideal 100 años, pero el código puede ser algo pesado en ejecutarse) de clima diario para las series de tiempo de temperatura máxima, mínima y precipitación de la pregunta "2)".
4) Revise cómo andan las autocorrelaciones de los datos generados, contra los observados, las correlaciones entre variables.
5) Grafique la variación estacional del promedio de la precipitación generada e histórica, además de la variación estacional de la desviación estándar de la precipitación diaria.
6) Mismos gráficos que la pregunta "5)", pero para temperatura máxima.
7) Mismos gráficos que la pregunta "5)", pero para temperatura mínima.

Código:

A Continuación, se desarrolla un código que resuelve las preguntas:

```
# El comando "rm()" borra elementos del Environment, en este caso toda
# la lista
rm(list=ls())
# El comando "graphics.off()" permite cerrar los gráficos abiertos.
graphics.off()

# Generamos una función "knn_generar", la cual genera clima en base al método
# k-nearest neighbor (k-NN). Este genera clima asumiendo la memoria de un
# instante de tiempo, para explícitamente considerar una memoria mayor, habría
# que modificar el código o las variables de entrada.
# Esta requiere de los siguientes datos de entrada:
# Inputs:
# V_D1: Punto de partida, consiste en un vector horizontal de valores que tenga
# información hidroclimática de todas las estaciones de interés.
# D2: Matriz 3D, que tiene los datos históricos que se utilizarán para generar
# series hidroclimáticas. En su primera dimensión tiene la escala temporal
# menor (por ejemplo, día), en su segunda dimensión, está la escala temporal
# mayor (suele ser años, los que son no periódicos) y en la tercera dimensión
# están las diferentes estaciones.
# w: Vector vertical con los pesos, que se le dará a cada variable en la
# estimación de las distancias Euclidianas.
# k: Número de valores análogos seleccionados, en el proceso de generación de
# variables hidroclimáticas. Es un parámetro del modelo, se aconseja tomar
# k=round(sqrt(n*(wind*2+1))), con "n" número de años en D2.
# wind: Ventana de tiempo que se considera para buscar análogos. Dado que las
# series hidroclimáticas son periódicas, hay que considerar una ventana de
```

```
# tiempo, para no comparar dos instantes de tiempo, que no tengan sentido físico.
# ID: Punto de partida dentro del año. Este indica dónde se encuentra "V_D1"
# dentro del período de un año (es la posición relativa del valor "V_D1" en la
# primera dimensión de D2). Por ejemplo, si consideramos años de 365 días en D2
# y "V_D1" corresponde al último día del año, entonces ID=365.
# N_gen: Número de valores que se generará. Notar que si se quiere generar 10
# años de datos y consideramos años de 365 días, tenemos que tener un
# N_gen=3650
#
# Outputs:
# Gen: Variables hidroclimáticas generadas, es una matriz 2D, cuya primera
# dimensión es la serie temporal y la segunda dimensión son las estaciones.
knn_generar=function(V_D1,D2,w,k,wind,ID,N_gen)
{
 # Se extrae el número de años dentro de "D2", lo cual está en la segunda
 # dimensión y se guarda en "n"
 n = dim(D2)[2]
 # Se extrae el número de estaciones dentro de "D2", lo cual está en la tercera
 # dimensión y se guarda en "m"
 m = dim(D2)[3]
 # Se genera un vector con "0" que tenga la dimensión de "k" y se guarda en
 # "pos"
 pos=array(0,c(k,1))

 # La variable "add_day" es una variable para poder transicionar entre años. Es
 # importante que se adopte un valor de add_day>wind, de lo contrario generara
 # errores. El motivo de adoptar add_day=30, es que se está considerando que
 # la variable "wind" no debiese ser más de 30.
 add_day = 30
```

```
# El comando "abind" permite combinar matrices de más de dos dimensiones
# (Notar, que el valor de along=2 indica que se concatenen las matrices en la
# segunda dimensión). En este caso en "D2_ini" se toman los diciembres del
# año anterior, dado que no se conoce el diciembre del año anterior al primer
# año, ese se rellena con valores de "0".
D2_ini = abind::abind(array(0,c(add_day,1,m)),D2[(365-add_day+1):365,1:(n-1),],along=2)
# En este caso en "D2_end" se toman los eneros del año siguiente, dado que no
# se conoce el enero del año de después del último año, ese se rellena con
# valores de "0".
D2_end = abind::abind(D2[1:add_day,2:n,],array(0,c(add_day,1,m)),along=2)
# Luego a la matriz "D2" se le agrega en la dimensión "1" diciembre del año
# anterior y enero del año siguiente. El motivo para hacer esto, es para los
# casos en que uno busque análogos a finales o inicios de un año, poder buscar
# dentro de la ventana "window", con facilidad.
D2_ex = abind::abind(D2_ini,D2,D2_end,along=1)

# Se genera un vector "i" que va de 1 a "k"
i=1:k
# Se suma todos los valores "1/i" y se guarda en "sumi". Esto sirve para
# estimar las probabilidades en el proceso de selección del k-NN
sumi=sum(1/i)
# Se inicializar el vector de probabilidades.
p=array(0,c(k,1))
# Se genera un ciclo "for" que estima las probabilidades "p[h]"
for (h in 1:k){
  p[h]=(1/h)/sumi
}

# Se genera la función de probabilidad acumulada
```

```
p=cumsum(p)


# Dado que se generarán varios días, en "Aux_V_D1" se irá guardando el día
# anterior. En la primera iteración "Aux_V_D1" toma el valor de "V_D1", para
# la siguiente iteración, toma el valor del nuevo día generado.
Aux_V_D1 = V_D1
# Si inicializa una matriz "Gen" que tendrá las series de tiempo generadas,
# esta tiene dimensiones "N_gen" x "m", siendo "N_gen" el número de días a
# generar y "m" el número de estaciones.
Gen = array(0,c(N_gen,m))
# Se genera un ciclo "for" con tantas iteraciones como variables a generar.
for (LL in 1:N_gen){
 # Se genera una variable "rim" la cual almacenará la distancia Euclidiana.
 # rim, es inicializado con valore "Inf" (infinito), con las dimensiones de
 # "D2_ex", en los períodos (días) y años. El inicializarla con valores "Inf"
 # permite descartar cualquier día fuera del rango de la ventana móvil de
 # análisis
 rim = array(Inf,dim(D2_ex)[1:2])
 # Generamos un "for" que recorra todos los potenciales días análogos.
 # Dado que la serie es periódica, se recorren los días en el rango desde
 # "ID-wind", hasta "ID+wind", dado que se ha agregado un mes a D2, para
 # generar D2_ex, es que se agrega además el valor "add_day". El recorrer en
 # vez de "D2_ex" completo en su primera dimensión (días), sino que solo la
 # ventana móvil tiene dos ventajas: 1) ahorra tiempo computacional, al no
 # hacer comparaciones contra días que no se parecerán; 2) permite mantener
 # la periodicidad de las series de tiempo.
 for (i in (ID-wind+add_day):(ID+wind+add_day)){
  # Se genera otro "for" el cual recorre los años de "D2_ex"
  for(j in 1:n){
```

```
    # Para cada valor de "D2_ex" en la ventana desde "ID-wind", hasta
    # "ID+wind", se resta contra "Aux_V_D1", estima su cuadrado y finalmente
    # se multiplica por el vector de peso "w", lo que suma los valores entre
    # estaciones o variables de "D2_ex". Estas distancias se almacenan en
    # "rim"
    rim[i,j] = (D2_ex[i,j,]-Aux_V_D1)^2%*%w
  }
}
# Esto para asegurarse de que no elija el último día, ni los días agregados
# al final. Se quitan los últimos 31 días de "rim", dando valore "Inf". El
# motivo de quitar los últimos 31 días, del último año, es que no se conoce
# el día siguiente de ellos.
rim[(dim(rim)[1]-add_day):dim(rim)[1],n]=Inf
# Además se quitan los primeros 30 días de "rim" del primer año (el motivo
# es que no se conocen esos días, fueron rellenados con valores "0")
rim[1:add_day,1]=Inf

# Se procede a seleccionar los "k" valores más pequeños dentro de "rim"
for (h in 1:k){
  # Se busca todas las coordenadas del mínimo "rim" (que tiene las
  # distancias Euclidianas). Esto quiere decir que "Aux2", tiene las
  # coordenadas donde están los mínimos
  Aux2=which(min(rim)==rim)
  if (length(Aux2)==1){
    # De haber un solo mínimo en Aux2, se guarda en "pos[h]"
    pos[h]=Aux2
    # Notar que "Aux2" es un único valor y no un par coordenado. Esto es
    # equivalente a re-ordenar "rim" a un vector en vez de matriz y estimar
    # la posición de dicho punto.
```

```
  }else{
    # En caso de haber más de un mínimo Aux2 (esto ocurrirá cuando hay dos
    # valores con exactamente las mismas distancias Euclidianes dentro de
    # Aux), se elige uno de manera aleatoria:
    pos[h]= sample(Aux2)[1]
  }
  # Dado que ya se guardó el mínimo, el valor de "Aux", es reemplazado por
  # infinito "Inf", lo cual hace que en el siguiente ciclo del "for", se
  # escoja el siguiente menor valor.
  rim[pos[h]]=Inf
}



# Se genera un aleatorio (uniforme entre 0 y 1) y se guarda en RN
RN=runif(1)
# Se recorre los valores de "h", entre 1 y "k"
for (h in 1:k){
  # Se recorre las probabilidades desde la mayor a la menor (subíndice
  # "k-h+1", hace que el "for" se recorra de "k" a 1)
  # Mientras RN, sea menor a la probabilidad p[k-h+1], se entrará el "if"
  # el minuto que dicha condición deja de ser cierta, no se entra al "if".
  # La última vez que se entra al "if" se deja guardada la posición "pos[]"
  # Que tiene el valor escogido aleatoriamente dentro de los "k" previamente
  # seleccionados.
  if (RN<=p[k-h+1]){
    Y=pos[k-h+1]
  }
}
# Dado que se quiere el día siguiente al elegido, "Index" que tiene la
```

```
# posición que usaremos, es "Y+1"
Index = Y+1
# Dado que "Index" tiene la posición del elemento que queremos guardado en
# un único número, hay que extraer de él, las coordenadas de interés.
# La primera coordenada, es el resto de la división entre "Index" y la
# primera dimensión de "D2_ex"
i = Index%%dim(D2_ex)[1]
# En aquellos casos en que el resto de la división Index y dim(D2_ex)[1] sea
# "0", "i" toma el valor de 0, pero debiese tomar el valor "dim(D2_ex)[1]",
# motivo por el que se agrega el siguiente "if":
if (i==0){
  i= dim(D2_ex)[1]
}
# La segunda coordenada "j", se obtiene de la división entre "Index" y la
# primera dimensión de D2_ex y redondearlo hacia arriba.
j = ceiling(Index/dim(D2_ex)[1])
# El nuevo valor generado es la coordenada "[i,j,]" de D2_ex. Con ello se
# traspasa los valores de todas las estaciones.
Gen[LL,] = D2_ex[i,j,]
# Se cambiar el valor de "Aux_V_D1" al nuevo valor "Gen[LL,]" recientemente
# obtenido.
Aux_V_D1 = Gen[LL,]
# Al "ID" se le suma 1, dado que se avanza 1 instante de tiempo. Cuando, se
# haya recorrido todo el año este pasa a ser superar a la primera dimensión
# de D2 (dim(D2)[1]), en una unidad, en cuyo caso, se reinicializa dándole
# el valor ID=1.
ID = ID+1
if (ID==(dim(D2)[1]+1)){
  ID=1
```

```
   }
  }
  # Por último, se devuelve el valor de "Gen"
  return(Gen)
}
```

```
# Fijamos el directorio donde tenemos el archivo Excel con los datos
setwd("C:/Users/HP/Dropbox/CC/0. CRISTIAN/0 New/UAI/Diplomado/2023/Clases/PPTs/Cristian_Chadwick/Hidrologia Estocastica en un Escenario Estacionario/k_NN_Generacion/Code")

# Con library() activamos la librería para abrir y cerrar archivos
# Excel
library(openxlsx)

# Leemos los archivos "CSV" con el comando "read.csv()"
# el archivo se carga como un data frame.
# Para la temperatura, importaremos temperaturas mensuales máximas y mínimas
# Partiremos con "Estero Arrayan En la Montosa"
Pr <- read.csv("Data/precip_cr2met_day.csv")
Tmax <- read.csv("Data/tmax_cr2met_day.csv")
Tmin <- read.csv("Data/tmin_cr2met_day.csv")

# Dado que las series de tiempo de "Pr", "Tmax" y "Tmin" tienen años bisiestos
# con 29 de Febrero, es necesario quitar dichos días, para facilitar el manejo
# de los datos.
# Se busca todos aquellos días 29 (columna 4 de Pr, tiene los días)
Dia_29 <- which((Pr[,4]==29))
```

```
# Luego, se busca todos aquellos casos en que el 29 fue en mes "2":
AUX <- which(Pr[Dia_29,3]==2)
# Luego se busca aquellos días 29 (Dia_29), cuyos meses eran "2":
Feb_29 <- Dia_29[AUX]
# Teniendo los índices, se genera un "Aux_Pr", "Aux_Tmax" y "Aux_Tmin", que se
# les quita los 29 de Febrero (esto por medio de evaluar en [-Feb_29,])
Aux_Pr <- Pr[-Feb_29,]
Aux_Tmax <- Tmax[-Feb_29,]
Aux_Tmin <- Tmin[-Feb_29,]
# Luego se procede a quitar los días del año "2020"
Y_20 <- which(Aux_Pr[,2]==2020)
# Las variables "Pr_d", "Tmax_d" y "Tmin_d" se quedan con años de 365 días entre
# el año 1979-2019 (inclusive)
Pr_d <- Aux_Pr[-Y_20,5]
Tmax_d <- Aux_Tmax[-Y_20,5]
Tmin_d <- Aux_Tmin[-Y_20,5]

# "n" es el número de años:
n <- length(Pr_d)/365

# Se agrupan las variables climáticas en una matriz en 3D:
# Meses x Años x Estación
# Es importante el orden de las dimensiones.
D2 <-array(c(Pr_d,Tmax_d,Tmin_d),c(365,n,3))

# Se puede verificar que D2 haya sido bien construida con:
# max(D2[,,1]-array(Pr_d,c(365,n)))
# min(D2[,,1]-array(Pr_d,c(365,n)))
# max(D2[,,2]-array(Tmax_d,c(365,n)))
```

```
# min(D2[,,2]-array(Tmax_d,c(365,n)))
# max(D2[,,3]-array(Tmin_d,c(365,n)))
# min(D2[,,3]-array(Tmin_d,c(365,n)))
# Esto debiese dar valores "0"

# Se genera un vector de pesos "w", cuyo peso es el inverso de la desviación
# estándar. Es importante, que sea un vector columna:
w <-array(c(1/sd(Pr_d),1/sd(Tmax_d),1/sd(Tmin_d)),c(3,1))
# Dado que tenemos Temperatura Máxima y Mínima, le estamos dando más
# peso a la temperatura que a la precipitación. Si queremos que tengan el
# mismo peso, se puede probar duplicando el peso de la precipitación.

# Se normaliza el vector "w":
w <-w/norm(w)

# Se asumen ventanas de 15 días que se revisarán (15 días antes y después del
# día en que se está, dejando una ventana de 31 días a revisar). Esta ventana
# es un parámetro del modelo a calibrar. En este caso se asumió 15, pero podría
# tener otro valor.
wind <- 15
# ID, en el día inicial, se puede tomar cualquier día del año para
# inicializarlo, pero tiene que ser consistente con "V_D1". Dado que tomaremos
# el último día de la serie de "D2", ese es el 31 de diciembre del 2019, es
# decir, el ID=365
ID <-365

# Los días que se generarán, en este caso, 100 años de 365 días. Mucho cuidado,
# esto ya es pesado y puede que colapse su computador, toma varios segundos en
# correr.
```

```
N_gen <- 365*100


# Se estima el valor de "k" con la ecuación heurística raíz del número de
# valores a considerar en la comparación. Dado que la ventana móvil a considera
# es "wind*2+1" y el número de años de D2 es "n" se toma la raíz de
# "n*(wind*2+1)"
k <-round(sqrt(n*(wind*2+1)))

# Se toma un día (puede ser cualquiera), para inicializar el generador k-NN.
# En este caso se toma el último día de D2 (cuidado con que el ID sea el mismo
# día del año)
V_D1 <- D2[365,n,]

# Se utiliza "knn_generar" para generar clima buscando análogos:
Gen <- knn_generar(V_D1,D2,w,k,wind,ID,N_gen)

# Se extraen en tres vectores los valores de "Gen" generados: "Pr_gen" para
# precipitaciones, "Tmax_gen" para temperatura máxima y "Tmin_gen" para
# temperatura mínima
Pr_gen <-array(Gen[,1],c(365,dim(Gen)[1]/365))
Tmax_gen <-array(Gen[,2],c(365,dim(Gen)[1]/365))
Tmin_gen <-array(Gen[,3],c(365,dim(Gen)[1]/365))

# Se comparan autocorrelaciones, entre las series generadas y las observadas
lag <- 1
cor(Gen[1:(dim(Gen)[1]-lag),1],Gen[(1+lag):(dim(Gen)[1]),1])
cor(Pr_d[1:(length(Pr_d)-lag)],Pr_d[(1+lag):(length(Pr_d))])
```

```
lag <- 2
cor(Gen[1:(dim(Gen)[1]-lag),1],Gen[(1+lag):(dim(Gen)[1]),1])
cor(Pr_d[1:(length(Pr_d)-lag)],Pr_d[(1+lag):(length(Pr_d))])


lag <- 1
cor(Gen[1:(dim(Gen)[1]-lag),2],Gen[(1+lag):(dim(Gen)[1]),2])
cor(Tmax_d[1:(length(Tmax_d)-lag)],Tmax_d[(1+lag):(length(Tmax_d))])

lag <- 2
cor(Gen[1:(dim(Gen)[1]-lag),2],Gen[(1+lag):(dim(Gen)[1]),2])
cor(Tmax_d[1:(length(Tmax_d)-lag)],Tmax_d[(1+lag):(length(Tmax_d))])

lag <- 3
cor(Gen[1:(dim(Gen)[1]-lag),2],Gen[(1+lag):(dim(Gen)[1]),2])
cor(Tmax_d[1:(length(Tmax_d)-lag)],Tmax_d[(1+lag):(length(Tmax_d))])

lag <- 1
cor(Gen[1:(dim(Gen)[1]-lag),3],Gen[(1+lag):(dim(Gen)[1]),3])
cor(Tmin_d[1:(length(Tmin_d)-lag)],Tmin_d[(1+lag):(length(Tmin_d))])

lag <- 2
cor(Gen[1:(dim(Gen)[1]-lag),3],Gen[(1+lag):(dim(Gen)[1]),3])
cor(Tmin_d[1:(length(Tmin_d)-lag)],Tmin_d[(1+lag):(length(Tmin_d))])

lag <- 3
cor(Gen[1:(dim(Gen)[1]-lag),3],Gen[(1+lag):(dim(Gen)[1]),3])
cor(Tmin_d[1:(length(Tmin_d)-lag)],Tmin_d[(1+lag):(length(Tmin_d))])
```

Se comparan correlaciones entre variables:

```
cor(Gen)

cor(cbind(Pr_d,Tmax_d,Tmin_d))
```

Se generan los gráficos:

```
plot(apply(D2[,,1],1,mean),type='l',lty=1,col=rgb(0.8,0.2,0.3),lwd=2,ylim = c(0,10),ylab = "",xlab = 'Meses',main = 'Promedio')

title(ylab = "Promedio Precipitación Diaria [°C]", line = 2)

matplot(apply(Pr_gen,1,mean),add=T,type='l',lty=2,col=rgb(0.3,0.8,0.3),lwd=2)

legend(x = 220, y = 9.9,legend = c('Histórico','Generado'), lty=1:2,
col=c(rgb(0.8,0.2,0.3),rgb(0.3,0.8,0.3)),lwd=c(2,2))


plot(apply(D2[,,1],1,sd),type='l',lty=1,col=rgb(0.8,0.2,0.3),lwd=2,ylim = c(0,18),ylab = "",xlab = 'Meses',main = 'Desv. Est.')

title(ylab = "Dev. Est. Precipitación Diaria [°C]", line = 2)

matplot(apply(Pr_gen,1,sd),add=T,type='l',lty=2,col=rgb(0.3,0.8,0.3),lwd=2)

legend(x = 220, y = 17.9,legend = c('Histórico','Generado'), lty=1:2,
col=c(rgb(0.8,0.2,0.3),rgb(0.3,0.8,0.3)),lwd=c(2,2))


plot(apply(D2[,,2],1,mean),type='l',lty=1,col=rgb(0.8,0.2,0.3),lwd=2,ylim = c(8,30),ylab = "",xlab = 'Meses',main = 'Promedio')

title(ylab = "Promedio Tmax Diaria [°C]", line = 2)

matplot(apply(Tmax_gen,1,mean),add=T,type='l',lty=2,col=rgb(0.3,0.8,0.3),lwd=2)

legend(x = 1, y = 14.2,legend = c('Histórico','Generado'), lty=1:2,
col=c(rgb(0.8,0.2,0.3),rgb(0.3,0.8,0.3)),lwd=c(2,2))


plot(apply(D2[,,2],1,sd),type='l',lty=1,col=rgb(0.8,0.2,0.3),lwd=2,ylim = c(0,5),ylab = "",xlab = 'Meses',main = 'Desv. Est.')

title(ylab = "Est. Desv. Tmax Diaria [°C]", line = 2)

matplot(apply(Tmax_gen,1,sd),add=T,type='l',lty=2,col=rgb(0.3,0.8,0.3),lwd=2)
```

```
legend(x = 150, y = 1.7,legend = c('Histórico','Generado'), lty=1:2,
col=c(rgb(0.8,0.2,0.3),rgb(0.3,0.8,0.3)),lwd=c(2,2))



plot(apply(D2[,,3],1,mean),type='l',lty=1,col=rgb(0.8,0.2,0.3),lwd=2,ylim = c(0,20),ylab = "",xlab =
'Meses',main = 'Promedio')

title(ylab = "Promedio Tmin Diaria [°C]", line = 2)

matplot(apply(Tmin_gen,1,mean),add=T,type='l',lty=2,col=rgb(0.3,0.8,0.3),lwd=2)

legend(x = 150, y = 19,legend = c('Histórico','Generado'), lty=1:2,
col=c(rgb(0.8,0.2,0.3),rgb(0.3,0.8,0.3)),lwd=c(2,2))


plot(apply(D2[,,3],1,sd),type='l',lty=1,col=rgb(0.8,0.2,0.3),lwd=2,ylim = c(0,3),ylab = "",xlab =
'Meses',main = 'Desv. Est.')

title(ylab = "Desv. Est. Tmin Diaria [°C]", line = 2)

matplot(apply(Tmin_gen,1,sd),add=T,type='l',lty=2,col=rgb(0.3,0.8,0.3),lwd=2)

legend(x = 150, y = 1.0,legend = c('Histórico','Generado'), lty=1:2,
col=c(rgb(0.8,0.2,0.3),rgb(0.3,0.8,0.3)),lwd=c(2,2))
```